

# Apache Kafka

## What is Apache Kafka?

- Is an open-source distributed, event streaming platform.
- Used to publish, store, and process streams of records in real-time

## Kafka Components

- **Topic:**
  - o is a named category of records
  - o logical category used to organize and store related messages
  - o producer send messages to the topic and consumers read messages from topic
  - o Kafka topics is divided into partitions which allows scalability and parallel processing on messages
  - o Topic is a collection of partitions
- **Partition**
  - o Partition is the smaller, ordered log inside a Kafka topic
  - o Kafka topic is divided into one or more partitions so Kafka can store and process large amounts of data efficiently
  - o The main reason of using partitions is parallelism and scalability
  - o Multiple consumers can process different partitions in parallel
  - o Note: ordering is guaranteed within a partition, not across the entire topic
- **Offset**
  - o An offset is a unique number that identifies the position of a message inside a Kafka partition
  - o Think of it like the index/position of a message
  - o The consumer uses the offset to remember where it is in the partition
  - o Offset allowing consumers to keep track of what they have read
- **Message**
  - o Is a piece of data/event that a producer sends to Kafka topic
  - o Think of it as a one of record of information
  - o Messages are stored in partitions
  - o Each message gets an offset within its partition
  - o Message can contain different types of data
- **Producers**
  - o Is an application or system that send messages/events to Kafka topic
  - o Producer doesn't send directly to the consumer, it sends messages/events to the Kafka topic, and consumers read messages/events from the Kafka topic

## - Consumers

- Is an application or system that reads messages/events from the Kafka topics
- Consumers knows its position using offset
- Consumer doesn't normally remove the messages from Kafka after reading it, that means another consumer can read the same message independently

## Kafka Commands

|                                                                           |                                                     |
|---------------------------------------------------------------------------|-----------------------------------------------------|
| <code>bin/kafka-topics.sh --help</code>                                   | Display available options and usage for the command |
| <code>bin/kafka-topics.sh --bootstrap-server localhost:9092 --list</code> | Is a command used for listing Kafka topics          |

## What a Kafka Producers?

- Write messages to Kafka topics, sometimes called publisher
- Messages send via producers are stored on Kafka for later use
- Can have many producers
- Producers can write to one or multiple topics as required

## Types of Producers

- Command-line
  - `kafka-console.producer.sh`
- in addition to the command-line tool, API libraries are available for Python, Java, and many other languages
- Kafka connect: is designed to interact with existing systems such as relational databases, and automatically transfer data between them

## kafka-console.producer.sh

- It is normally found in the Kafka bin directory, along with other Kafka tools
  - `bin/kafka-console-producer.sh`
- several command-line options, some required, others not
- required options
  - `--bootstrap-server` → specifies which server to use
    - `--bootstrap-server localhost:9092`
  - `--topic` → which requires the topic to write to
    - `--topic phishing-sites`
- Use Ctrl-C to exit
- Can also use `|` symbol to pass messages

- Example
  - `bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic testing`
  - once run, can enter our messages like down and use Ctrl+C to exit
    - > this is the first message
    - > This is the second message
    - > this is the third message
    - ctrl+c to exit
- Non-interactive example
  - `echo "this is the first message" | bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic testing``

## Kafka Consumers

- reads message from Kafka topics, sometimes called subscribers
- there can many consumers communicate with a Kafka system at a time
- consumers can read from one or multiple topics

## Types of Consumers

- many different kinds of consumers
- command-like
  - `kafka-console-consumer.sh`
- there are also components within Kafka connect that allow users to read data from Kafka topics

## kafka-console-consumer.sh

- it is normally found in the Kafka bin directory, along with other Kafka tools
  - `bin/kafka-console-consumer.sh`
- several options, some required
  - `--topic`
  - `--bootstrap-server`
  - Use ctrl+c to exit
- Optional arguments
  - `--from-beginning`
    - Tells Kafka system to send all message in topic, even ones that may have been already read
  - `--max-messages <number>`
    - Used to read up to a maximum number of messages before exiting

## Interactive example

- `bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic testing --from-beginning`
  - o > this is the first message
  - o > this is the second message
  - o > this is the third message
  - o Ctrl + c

## Non-interactive example

- `bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic testing --from-beginning --max-messages 2`
  - o this is the first message
  - o this is the second message

## Kafka Architecture

- **Kafka server**
  - o A cluster of one or more computers
  - o Stores data
  - o Manages communications
  - o Can also integrate with other systems (databases, logs, ...)
- **Kafka client**
  - o Read via Kafka consumer
  - o Write via Kafka producer
  - o Process data as required

## Kafka server

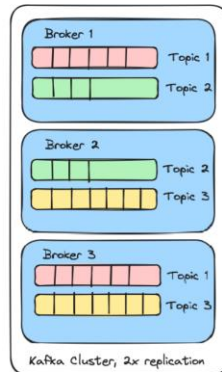
- The primary task of a Kafka server is acting as a Kafka broker
- Broker handles communication between consumers and producers
- The server also handles data storage
- Data written by producers is stored and organized via topics
- Topics are partitioned, meaning that they are stored in separate pieces
- Messages are retrieved in same order as written

## Partitions & replication

- Kafka is fault-tolerant, if one system goes offline, other can provide the requested data
- For each Kafka cluster, there is a replication factor

- Max number of failure -1 = replication factor
- Max replication factor == number of servers
- The fault-tolerance is handled by replication topic partitions within the cluster

### Example



- Kafka cluster with 3 brokers
- 3 topics defined
- 2x replication == can lose 1 system
- Broker 1 has copies of Topic 1 & Topic 2
- Broker 2 has copies of Topic 2 & Topic 3
- Broker 3 has copies of Topic 1 & Topic 3
- Each topic is shared across 2 brokers within the cluster

### What is Zookeeper?

- Zookeeper is a framework to manage information and provide services necessary for running distributed systems
- Zookeeper primarily used by developer to create distributed applications
  - Users interact with Zookeeper
- Example applications
  - Kafka
  - Hadoop
  - Neo4j

### What does zookeeper do?

- Zookeeper provides services necessary to run distributed applications
  - Handling configuration information
  - Naming systems to prevent conflicts
  - Provides the ability to synchronize across systems, such as determining what systems are available, when they should start, ...
- Zookeeper designed as a framework to prevent individual distributed applications from implementing custom versions of services

### Zookeeper and Kafka

- Kafka uses Zookeeper for cluster management
  - Newer versions of Kafka can use KRaft

- Kafka uses two primary configuration files for server /cluster
  - o config/zookeeper.properties
  - o config/server.properties
- the primary Kafka server configuration file is the config/server.properties file
  - o broker details
  - o network configuration
  - o event storage location
  - o basic topic configurations, including replication

## starting a Kafka cluster

- Kafka clusters are started in two pieces
  - o `bin/zookeeper-server-start.sh config/zookeeper.properties`
    - this start up the basic zookeeper server and configuration details found in the zookeeper.properties
  - o `bin/kafka-server-start.sh config/server.properties`
    - which will start the actual Kafka services as defined in the server.properties file

## stopping a Kafka cluster

- to stop the Kafka cluster, we do the reverse using
  - o `bin/kafka-server-stop.sh`
  - o `bin/zookeeper-server-stop.sh`
- note the reverse order of shutdown

## Kafka Topics

- topics are logical grouping of events
- topic is similar to table in relational database, where each event has a similar structure and meaning
- a topic is sometimes referred to as an event log, and messages written to the log are immutable
- messages in a topic can be read or written, but not modified

## Creating Topics

- multiple ways to create topics
- we'll use the `bin/kafka-topics.sh` script

`bin/kafka-topics.sh`

```
--bootstrap-server <server>
--topic <topic_name>
--create
```

- example
  - o `bin/kafka-topics.sh --bootstrap-server localhost:9092 --topic orders --create`

## Other Variations

- optional arguments for `bin/kafka-topics.sh --create`
  - o `--replication-factor <x>`
    - Define the replication factor of the topic
  - o `--partitions <x>`
    - Specify the number of partitions manually
  - o `--describe`
    - Get details about the topic configuration
  - o Example
    - `bin/kafka-topics.sh --bootstrap-server localhost:9092 --topic orders --create --replication-factor 3 --partitions 3`
  - o Example
    - `bin/kafka-topics.sh --bootstrap-server localhost:9092 --topic orders --describe`

## Removing Topics

- used to remove the topic from the Kafka server/cluster, and it's important note that also removes all messages in the topic
- example
  - o `bin/kafka-topics.sh --bootstrap localhost:9092 --topic orders --delete`

## Kafka Troubleshooting

- The finding and fixing problems that happens when using Apache Kafka

### Kafka Broker is not running

- You run the following example  

```
bin/kafka-console-producer.sh \  
  --bootstrap-server localhost:9092 \  
  --topic test
```
- You get:
  - o Connection to node -1 could not be established
  - o Problem: Kafka broker may not be running
  - o Check
    - `jps`
  - o then check port
    - `ss -lntp | grep 9092`

### Wrong Bootstrap Server

- you run
  - o `bin/kafka-console-consumer.sh --bootstrap-server localhost:9093 --topic test`
- problem wrong port
- fix
  - o `bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test`

### Topic Doesn't Exist

- you run
  - o `bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic sales`
- but sales doesn't exist
- check
  - o `bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --list`
  - o if sales doesn't exist there, create it

```
bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic sales --partitions 3 --replication-factor
```

## Consumer Doesn't Show Old Messages

- you produced the following
  - o Hello
  - o Kafka
  - o Message 3
- Then you start
  - o `bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test`
- you don't see the old messages
  - o problem: the consumer isn't necessary reading from the beginning
  - o fix it as following
    - `bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning`
  - o now you can read the existing messages from the beginning

## Port 9092 is already being used

- Kafka tries to start, but you get something like:
  - o Address already in use
- Check
  - o `sudo lsof -i :9092`
- you discover another process is using port 9092
- **problem:** port conflict
- **troubleshooting:** identify the process and either stop it or configure Kafka to use another port

## Kafka Start Then Stop

- you start Kafka, but it immediately shuts down
- you check the Kafka logs
  - o `tail -f logs/server.log`
- you might find
  - o Permission denied
- **Problem:** Kafka does not have permission to access log/data directory
- **Troubleshooting:** check ownership and permissions of the relevant directories